Responsive & Layouts

Agilent layout is optimized for a max-width of 1280px screen and designed for a fluid grid system, the Agilent.com website will adjust to any form-factor. All dimension (size, margin, padding, etc) should be in percentages as the width changes to maintain a fluid grid across any screen.

We want all of our designs to look great across all platforms. Sometimes that means making small tweaks at different screen sizes.

Best Practices

What Is Responsive Web Design?

Responsive web design follows the three main principles: fluid grids, responsive media, and media queries. In some cases, responsive web design also makes use of the media viewport meta tag when a device can't determine the initial width or scale of a website, which causes media queries to not trigger. Here are the basic responsive web design principles explained:

1. Fluid Grids

Fluid grids work like any other design grid - they allow you to arrange elements on a page in a visually attractive way. However, unlike a traditional grid, a fluid grid will resize based on the screen size and can adapt to any width because it uses relative units of measurement such as **percentages or em units**, instead of fixed units such as pixels.

2. Media Queries

Media queries allow you to get even more specific with your responsive design and adjust it accordingly based on a particular screen size. The websites use media queries to gather data that helps them determine the size of a screen and then load the appropriate CSS styles.

3. Responsive Media

The third core principle of responsive web design is responsive or flexible media. Given that modern websites use a lot of images, videos, and other media files, it’s imperative that those types of content respond to different screen sizes.

Normally, designers would include the image dimensions in their specs. But, this doesn’t work for responsive design due to fixed units of measurement mentioned above. Instead, you've got to make use of the max-width property for image files, videos, and other media types. To ensure media files don’t go past their container and scale nicely based on the screen size, the max-width property should be set to 100%.

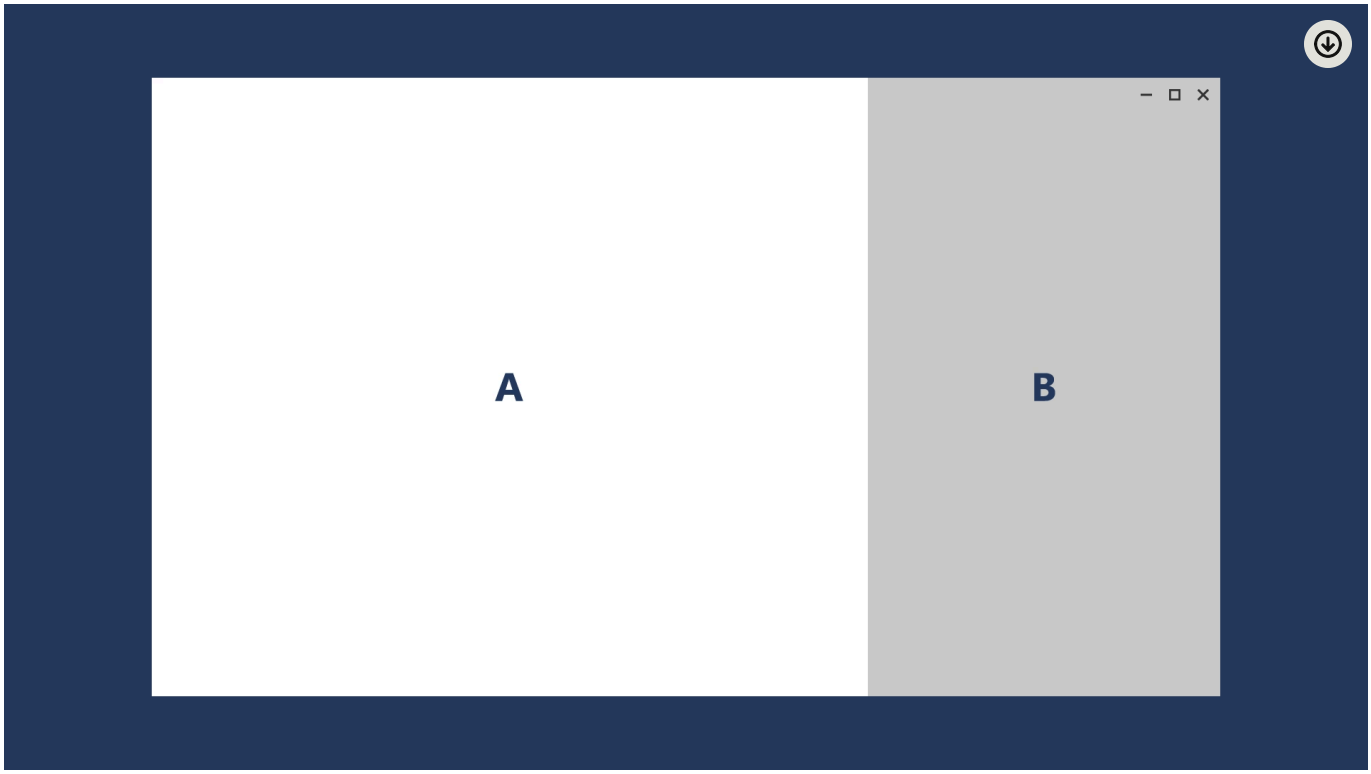
Responsive Web Resources:

- W3schools
- Material Design
- Bootstrap
- Demos of responsive sites

Media should never be stretched, and always scale appropriately.

Reposition

You can alter the location and position of UI elements to make the most of the window size. In this example, the smaller window stacks elements vertically. When the app translates to a larger window, elements can take advantage of the wider window width.



Resize

You can optimize for the window size by adjusting the margins and size of UI elements. For example, this could augment the reading experience on a larger screen by simply growing the content frame.

Reflow

By changing the flow of UI elements based on device and orientation, your app can offer an optimal display of content. For instance, when going to a larger screen, it might make sense to add columns, use larger containers, or generate list items in a different way.

This example shows how a single column of vertically scrolling content on a smaller screen that can be reflowed on a larger screen to display two columns of text.

Show/Hide

You can show or hide UI elements based on screen real estate, or when the device supports additional functionality, specific situations, or preferred screen orientations.

For example, media player controls reduce the button set on smaller screens and expand on larger screens. The media player on a larger window can handle far more on-screen functionality than it can on a smaller window.

Part of the reveal-or-hide technique includes choosing when to display more metadata. With smaller windows, it's best to show a minimal amount of metadata. With larger windows, a significant amount of metadata can be surfaced.

Replace

This technique lets you switch the user interface for a specific breakpoints. In this example, the nav pane and its compact, transient UI works well for a smaller screen, but on a larger screen, tabs might be a better choice.

Re-architect

You can collapse or fork the architecture of your app to better target specific devices. In this example, expanding the window shows the entire master/details pattern.

Breakpoints

These breakpoints describe column and width specifications for different screens and devices.

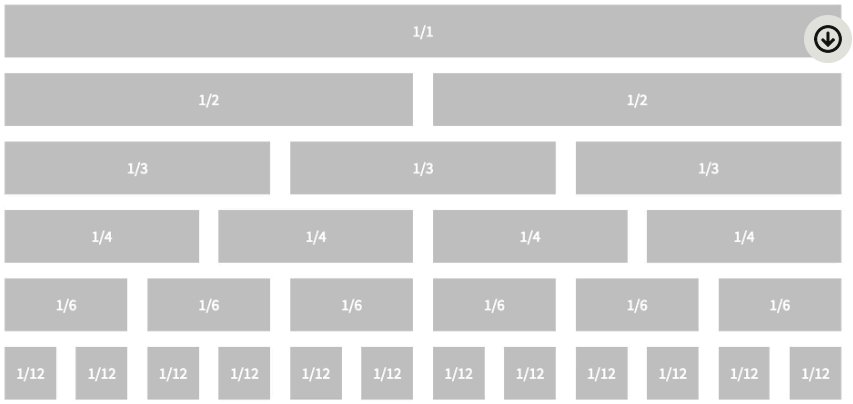
- Horizontal scroll bar should only appear when the breakpoints reaches below 719px
- Smaller windows below 719px are supported by WOMP Mobile on most Agilent.com pages
- Responsive breakpoint system is referenced from Material Design
- Preview material design breakpoint using an interactive resizer to view across desktop, mobile, and table

Breakpoint	**Window**	**Columns**
1024 px - 1400px +	Desktop layout	12
720 - 1023 px	Tablet in Portrait mode	12
0 - 719 px	Mobile	Womp or custom

To learn more about responsive layout grid, visit Google Material Design.

Layouts

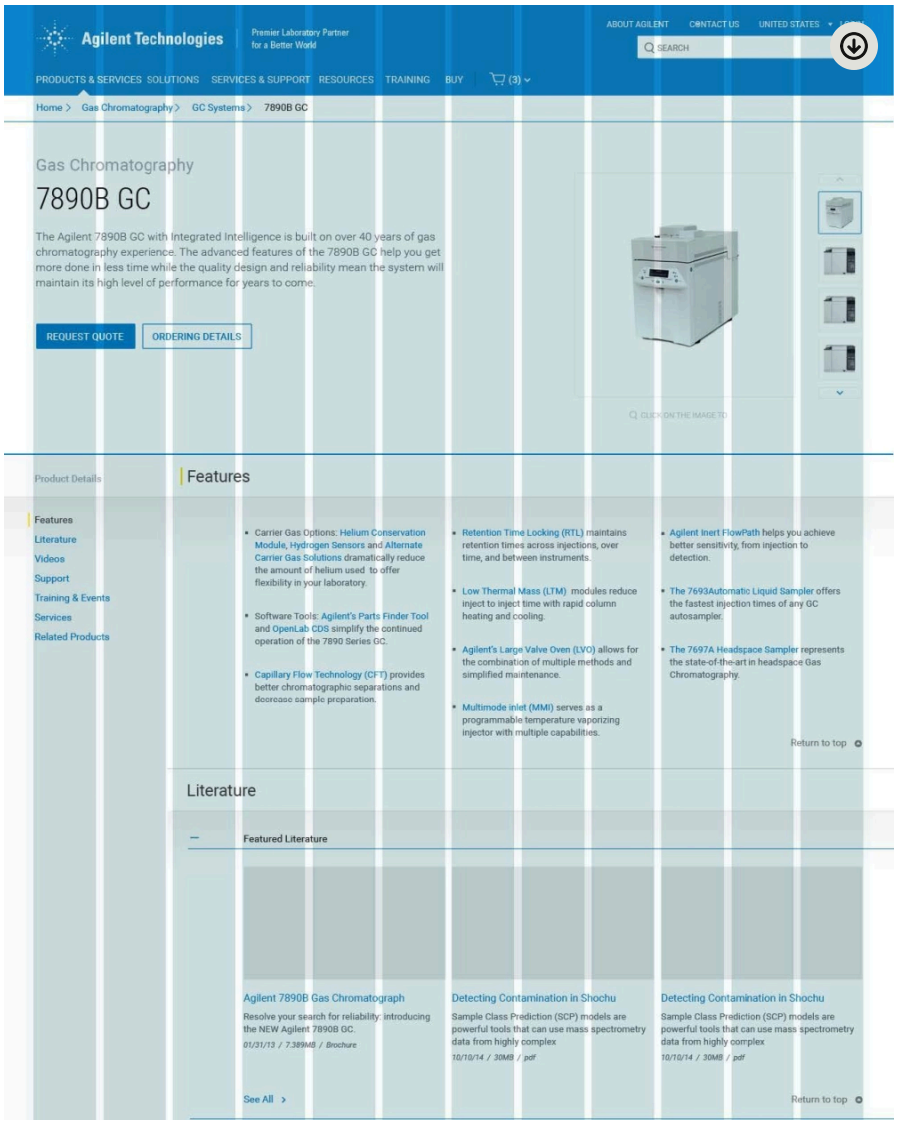
Column width is defined using percentages, rather than fixed values, to allow content to flexibly adapt to any screen size. The number of columns displayed in the grid is determined by the breakpoint range (a range of predetermined screen sizes) at which a screen is viewed, whether it’s a breakpoint for mobile, tablet, or another size.



This 12-column is an example of a fluid responsive grid that provides structure for website content.

Layout 1 (Legacy)

Layout 1 is used across many Agilent pages, include homepage, products, applications, search, cart, checkout, etc.



DESKOP

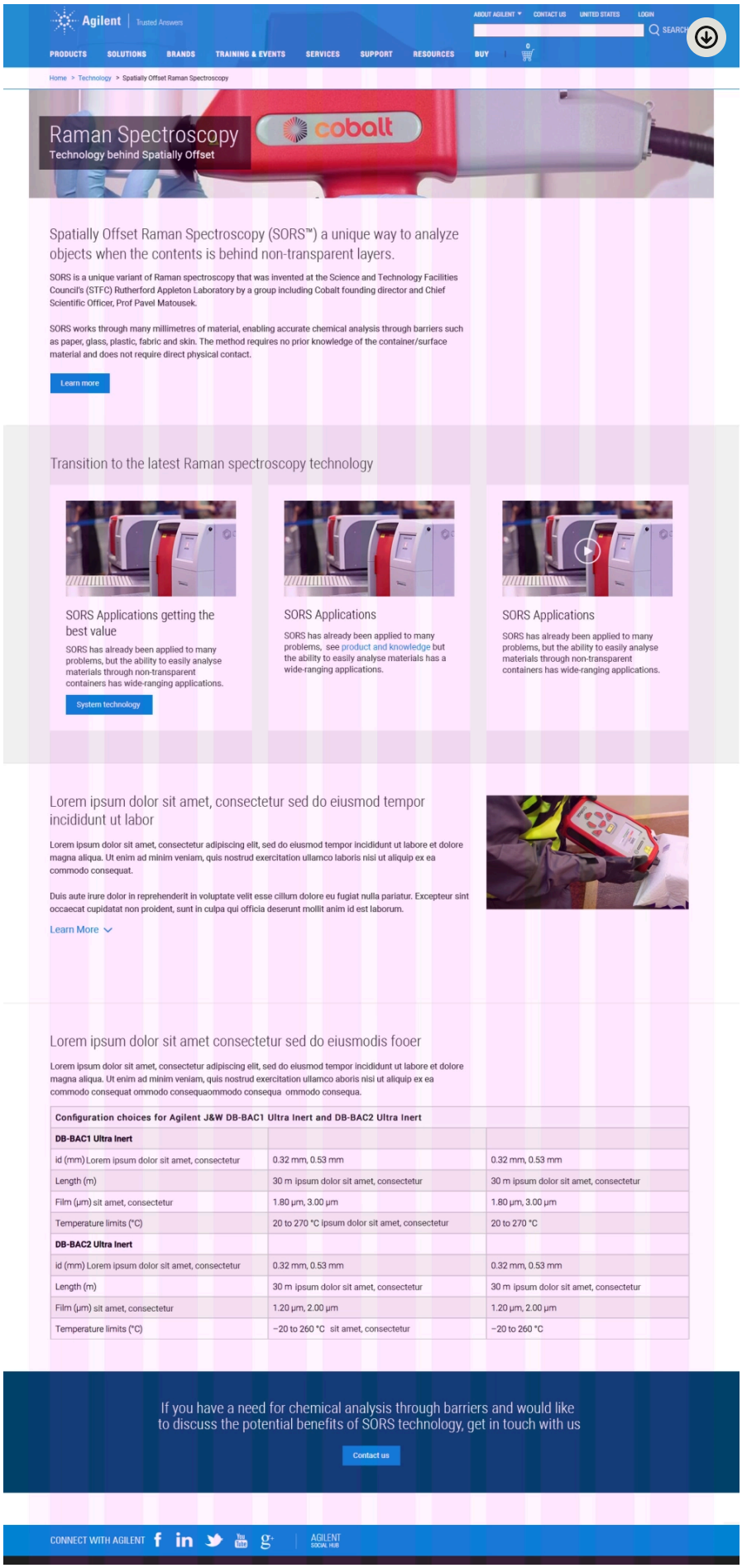
- Optimized for a max-width of 1280px
- 12 Column Grid System
- Master Container is 1190px
- Column is 90x (fluid)
- Column Margin is 10px

TABLET

- 12 Column Grid System
- Master Container is 994px
- Column Margin is 10px

Layout 2

Layout 2 is used across FAT and storytelling pages



DESKTOP

- Optimized for a max-width of 1280px
- 12 Column Grid System
- Master Container is 1194px
- Column is 72px (fluid)
- Column Margin is 30px
- Space between each component is 60px

TABLET

- 12 Column Grid System
- Master Container is 994px
- Column Margin is 10px

White space affects how the user focuses their attention on the content. It makes it easier to know what to read and where to begin. Spacing between typographic elements should be open enough to feel light, but close enough to establish a proper relationship between elements.

Header & Body Copy

- When setting headers and body copy, white space should be **60px, 30px, 20px, or 10px**

Web Spacing

- Spacing between elements/components, should be: **60px, 40px, 30px, 20px, 10px, 5px**

FOUNDATION
Iconography

FOUNDATION
Formatting

Last modified on Thu Aug 28 2025 8:03:47 AM